

Algorithms, AI & Data Science II

Prof. Dr. Ingo Scholtes

Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg
ingo.scholtes@uni-wuerzburg.de

Motivation

- ▶ we introduced Markov chain models for random walks in graphs
- ▶ in a motivating example, we used a Markov chain to calculate **visitation probabilities** of a drunk friend
- ▶ in simple examples, we found that the visitation probabilities reached **a stationary state**
- ▶ how can we calculate the **stationary state** of a Markov chain?
- ▶ when does a stationary state **exist** and how does it depend on the **initial state**?



Map of Würzburg

image credit: Map created from openstreetmap data

Notes:

- **Lecture 15: Markov Chain Convergence and PageRank** 30.06.2025
- **Educational objective:** We explore the existence and uniqueness of stationary states of Markov chains and explain how we can calculate them based on the eigenvalues and eigenvectors of transition matrices. We show how we can use stationary states of random walks with restarts to rank search results in the Web.
 - Stationary States of Markov Chains
 - Markov Chain Convergence Theorem
 - Random Walks with Restarts and PageRank
- **Exercise sheet 08:** Markov Chains and Random Walks due 07.07.2024
- **Handout version:** 2025/06/24 14:47:39

Notes:

Reminder: visitation probabilities

- ▶ let $\pi^{(t)} = (\pi_1^{(t)}, \pi_2^{(t)}, \dots)$ be **probabilities of states** (visitation probabilities) at time t
- ▶ we call $\pi^{(0)}$ **initial state** of the Markov chain

example

$$\pi^{(0)} = (1, 0, 0, 0, 0, 0, 0)$$

$$\pi^{(1)} = \left(0, \frac{1}{2}, \frac{1}{2}, 0, 0, 0, 0\right) = \pi^{(0)} \cdot \mathbf{T}$$

$$\begin{aligned} \pi^{(2)} &= \left(\frac{5}{12}, \frac{1}{4}, \frac{1}{6}, \frac{1}{6}, 0, 0, 0\right) \\ &= \pi^{(1)} \cdot \mathbf{T} = \pi^{(0)} \cdot \mathbf{T}^2 \end{aligned}$$

$$\mathbf{T} = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 & 0 \\ \frac{1}{3} & 0 & \frac{1}{3} & \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ 0 & 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ 0 & 0 & 0 & \frac{1}{3} & \frac{1}{3} & 0 & \frac{1}{3} \\ 0 & 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \end{bmatrix} \end{bmatrix}$$

using initial state $\pi^{(0)}$ and transition matrix $\mathbf{T}$ we can calculate **state $\pi^{(t)}$ for any time t** as $\pi^{(t)} = \pi^{(0)} \cdot \mathbf{T}^t$

Stationary States of Markov chains

- ▶ in our example network, we observe

$$\delta(\pi^{(t)}, \pi^{(t+1)}) \rightarrow 0 \text{ for } t \rightarrow \infty$$

- ▶ in this example, Markov chain eventually reaches **stationary state**

$$\pi := \lim_{t \rightarrow \infty} \pi^{(t)} \cdot \mathbf{T}^t$$

- ▶ after sufficiently long time, **it does not matter** at which time t we calculate visitation probabilities $\pi^{(t)}$
- ▶ allows us to make statements about **long-term behavior** of Markov chains (and random walks)
- ▶ given transition matrix, how can we calculate **stationary state** of a Markov chain?

$$\mathbf{T} = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 & 0 \\ \frac{1}{3} & 0 & \frac{1}{3} & \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ 0 & 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ 0 & 0 & 0 & \frac{1}{3} & \frac{1}{3} & 0 & \frac{1}{3} \\ 0 & 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \end{bmatrix} \end{matrix}$$

$\pi = ?$

Notes:

Notes:

Eigenvalues and eigenvectors

- ▶ consider a square real-valued matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$
- ▶ for $x \in \mathbb{R}^n$ we have $x \cdot \mathbf{A} \in \mathbb{R}^n$
- ▶ matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ can be viewed as **linear transformation** that maps every point $x \in \mathbb{R}^n$ to (possibly different) point $x' := x \cdot \mathbf{A} \in \mathbb{R}^n$
- ▶ we call $\lambda \in \mathbb{C}$ an **eigenvalue** of $\mathbf{A}$ for **eigenvector** $x \in \mathbb{C}^n$ iff

$$x \cdot \lambda = x \cdot \mathbf{A}$$

- ▶ eigenvector x with eigenvalue λ is a vector for which **linear transformation only scales** x by factor λ

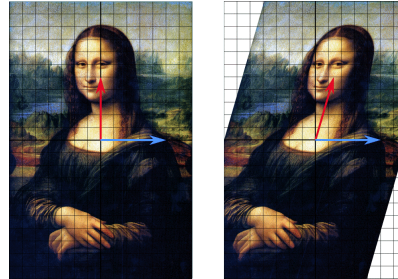


image credit: Wikimedia Commons, User TreyGreer62, CC-BY-SA

example

$$\mathbf{A} = \begin{pmatrix} 1 & \lambda \\ 0 & 1 \end{pmatrix}$$

performs **shear parallel to x-axis**
direction in which $\mathbf{A}$ only “stretches” plane $\mathbb{R}^2$ by factor λ is eigenvector of $\mathbf{A}$ with eigenvalue λ

Spectrum of a matrix

- ▶ for given matrix $\mathbf{A}$ we can **calculate eigenvalues and eigenvalues** by finding solutions to following equation

$$x\lambda = x \cdot \mathbf{A}$$

- ▶ for $\mathbf{A} \in \mathbb{R}^{n \times n}$ we can have n solutions, i.e. we can have eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n \in \mathbb{C}$ with associated eigenvectors $v_1, \dots, v_n \in \mathbb{C}^n$
- ▶ eigenvectors associated with **distinct eigenvalues** are **linearly independent**
- ▶ if same eigenvalue λ appears with different linearly independent eigenvectors we call λ **degenerate**
- ▶ (ordered) sequence of eigenvalues λ_i is called **spectrum of matrix $\mathbf{A}$**

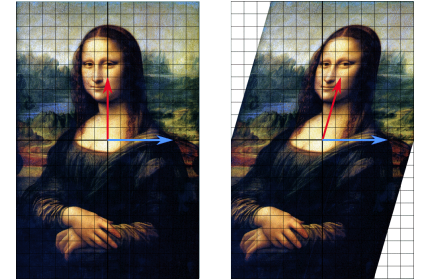


image credit: Wikimedia Commons, User TreyGreer62, CC-BY-SA

$$\mathbf{A} = \begin{pmatrix} 1 & \lambda \\ 0 & 1 \end{pmatrix}$$

$$\lambda_1 = \lambda_2 = 1 \\ v_1 = (1, 0), v_2 = (-1, 0)$$

Notes:

Notes:

Stationary state of Markov chains

- ▶ in our example $\pi^{(t)}$ ceases to change, i.e. process reaches **stationary state** $\pi := \lim_{t \rightarrow \infty} \pi^{(t)}$ such that

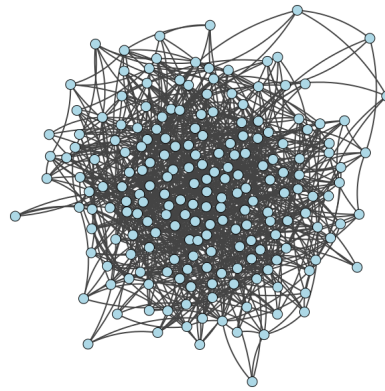
$$\pi = \pi \cdot \mathbf{T}$$

- ▶ this is an **eigenvalue problem** of the form

$$\pi \cdot \lambda = \pi \cdot \mathbf{T}$$

with eigenvalue $\lambda = 1$

- ▶ we can calculate stationary state as **left eigenvector of transition matrix corresponding to eigenvalue $\lambda = 1$**



convergence to stationary state for random walk in example graph

Stationary state of Markov chains

- ▶ stationary states are **important property of Markov chains**
- ▶ enables us to make statements about a system's "equilibrium state" in the **limit of infinite time**
- ▶ for Markov chains modelling random walks in graphs, we can use stationary states to define **node centrality**

open questions

- ▶ does Markov chain reach a stationary state for any initial state? (**existence**)
- ▶ are there different stationary states for different initial states? (**uniqueness**)



Google Search

I'm Feeling Lucky

Google.ch offered in: [Deutsch](#) [Français](#) [Italiano](#) [Rumantsch](#)

image credit: screenshot of Google search, self-created

random-walk based centrality

- ▶ let random walker "explore" a network
- ▶ we expect "important" nodes to be visited more frequently
- ▶ for stationary state π define centrality of node v as π_v

Notes:

Notes:

Practice Session

- ▶ we use eigenvectors to calculate the stationary state of Markov chains
- ▶ we explore the existence and uniqueness of stationary states

14-01 - Total Variation Distance and Stationary States of Markov Chains

June 14, 2023
 Hugo Schuster

Today we will calculate the total variation distance between stationary probabilities. We further discuss ergodicity and ergodicity of the transition matrix to calculate the stationary state of a Markov chain.

```

1 import numpy as np
2 import numpy.linalg as nla
3
4 import random as rnd
5 import matplotlib.pyplot as plt
6
7 from my_utils import loading as u
8
9 import time as t
10
11 plt.rcParams["figure.figsize"] = (10, 10)
12
13 def main():
14     # ...

```

We reuse the functions and the example from the previous notebook:

```

1 def transition_matrix(network, weights):
2     # We first create an adjacency matrix that also contains weights
3     # as a square matrix of size n x n
4     n = network.number_of_nodes()
5     # We create a weighted sum degree for the purposes of normalization
6     sum_degrees = sum(weights.values())
7     # We create an empty matrix of size n x n
8     A = np.zeros((n, n))
9     # We calculate for each node different range of nodes in the following loop
10     for node in network.nodes():
11         # We calculate the weighted sum of edges to network number of nodes:
12         # We calculate the weighted sum of edges to network number of nodes:
13         A[node] = weights.values() / sum_degrees
14     return A
15
16 def calculate_stationary_probabilities(network, initial_state, A):
17     # We calculate the stationary probabilities using the power method
18     # We calculate the stationary probabilities using the power method
19     return A

```

practice session

see notebook 14-01 in gitlab repository at

→ https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2

Existence of stationary state?

- ▶ for a given Markov chain, **does stationary state exist** for **any** initial state $\pi^{(0)}$?

example: random walk in directed ring lattice

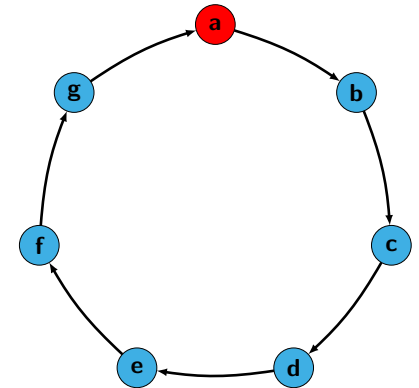
$$\pi^{(0)} = (1, 0, 0, 0, 0, 0, 0)$$

$$\pi^{(1)} = (0, 1, 0, 0, 0, 0, 0)$$

$$\pi^{(999)} = (0, 0, 0, 0, 0, 1, 0)$$

$$\pi^{(1000)} = (0, 0, 0, 0, 0, 0, 1)$$

- ▶ for Markov chain modelling random walk in directed ring lattice, we observe **periodic** behaviour
- ▶ stationary state is only reached for initial state $\pi^{(0)} = \left(\frac{1}{7}, \frac{1}{7}, \dots, \frac{1}{7}\right)$ but not for **any initial state**



example

random walk does not reach a stationary state for $\pi^{(0)} = (1, 0, \dots, 0)$

Notes:

Notes:

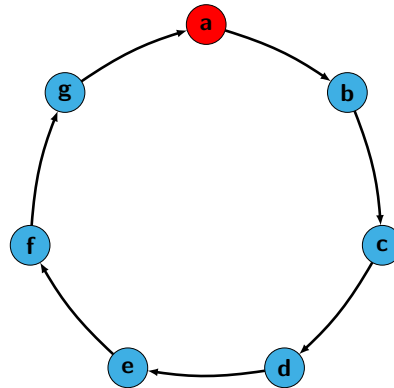
Aperiodicity

- ▶ let $\gcd\{a_1, \dots, a_n\}$ be the **greatest common divisor** of numbers $a_1, \dots, a_n$
- ▶ for state i of Markov chain, we define **period** $r(i)$ of i as

$$r(i) := \gcd\{t \geq 1 : (\mathbf{T}^t)_{ii} > 0\}$$

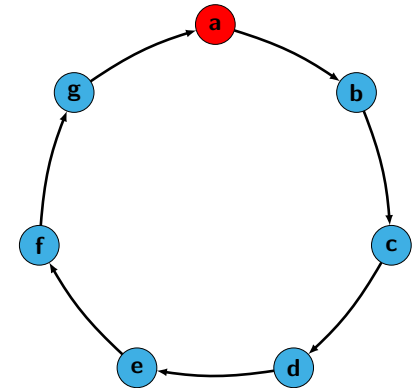
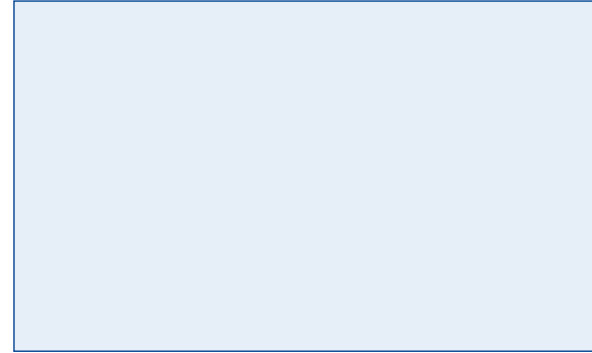
i.e. gcd of times t at which we can return to state i

- ▶ we call transition matrix **aperiodic** iff $r(i) = 1 \forall i \in V$
- ▶ transition matrix is **periodic** if $\exists i \in V : r(i) \neq 1$



Group Exercise 15-01

- ▶ Consider a Markov chain model of a random walk in the graph shown on the right. Calculate the period of all states and determine whether the corresponding transition matrix $\mathbf{T}$ is aperiodic.



Notes:

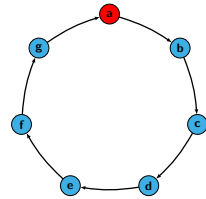
Notes:

Lazy Markov chains

- ▶ can we **make random walk in a graph aperiodic?**
- ▶ include non-zero probability $\epsilon > 0$ to **stay in current state**, i.e. $T_{ii} = \epsilon$ for all states i
- ▶ we obtain **lazy Markov chain**, where no transition to other state occurs with probability ϵ
- ▶ random walk in graph with **self-loops**
- ▶ for **period of all states** i we then have

$$r(i) = \{1, t_1, t_2, t_3, \dots\} = 1$$

i.e. **lazy Markov chains have aperiodic transition matrices**



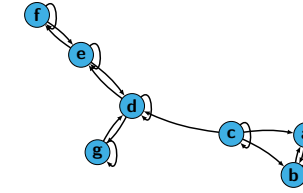
$$\mathbf{T} = \begin{matrix} & \begin{matrix} a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \\ e \\ f \\ g \end{matrix} & \begin{bmatrix} \frac{1}{100} & \frac{99}{100} & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{100} & \frac{99}{100} & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{100} & \frac{99}{100} & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{100} & \frac{99}{100} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{100} & \frac{99}{100} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{100} & \frac{99}{100} \\ \frac{99}{100} & 0 & 0 & 0 & 0 & 0 & \frac{1}{100} \end{bmatrix} \end{matrix}$$

example

by setting $T_{ii} > 0$ for all $i \in V$ we can make $\mathbf{T}$ aperiodic

Uniqueness of stationary state?

if a stationary state exists, is it **unique** for all initial states $\pi^{(0)}$?



example 1

random walk starts in node a , i.e.

$$\pi^{(0)} = (1, 0, 0, 0, 0, 0, 0)$$

$$\pi^{(50)} \approx \left(\frac{1}{2}, \frac{1}{2}, 0, 0, 0, 0, 0\right)$$

example 2

random walk starts in node g , i.e.

$$\pi^{(0)} = (0, 0, 0, 0, 0, 0, 1)$$

$$\pi^{(50)} \approx \left(0, 0, 0, \frac{3}{10}, \frac{3}{10}, \frac{2}{10}, \frac{2}{10}\right)$$

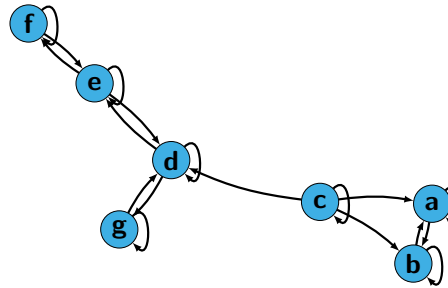
this Markov chain has **two stationary states** (i.e. eigenvalue $\lambda = 1$ is degenerate and appears with two linearly independent eigenvectors)

Notes:

Notes:

Irreducibility

- ▶ we say that transition matrix $\mathbf{T}$ is **reducible** iff there is a $\delta \geq 0$ such that $(\mathbf{T}^t)_{ij} = 0$ for a pair i, j and all $t > \delta$
- ▶ otherwise, we call $\mathbf{T}$ **irreducible**
- ▶ for Markov chain capturing random walk in graph, irreducibility of $\mathbf{T}$ corresponds to **(strong) connectivity of graph** → AKIDS 1
- ▶ transition matrix of random walk in any **directed and strongly-connected graph** is irreducible
- ▶ transition matrix of random walk in any **undirected connected graph** is irreducible



example for reducible transition matrix for weakly connected network

Markov chain convergence theorem

Markov chain convergence theorem

Let $\mathbf{T}$ be an **irreducible and aperiodic transition matrix** of a Markov chain and let $\pi^{(0)}$ be an **arbitrary initial state**.

For $\pi^{(t)} := \pi^{(0)} \cdot \mathbf{T}^t$ there exists **exactly one** stationary state π such that

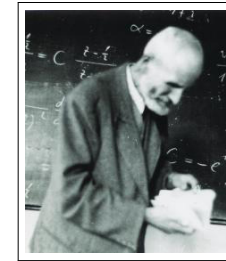
$$\delta(\pi^{(t)}, \pi) \rightarrow 0 \quad (t \rightarrow \infty)$$

- ▶ follows from **Perron-Frobenius theorem** for real-square matrices with positive entries

→ O Perron, 1907 → G Frobenius, 1912

observation

- ▶ if conditions of theorem are satisfied the stationary state π is **independent from initial state**
- ▶ over time, Markov chain **“forgets” state in which it started**



Oskar Perron
1880 – 1975

image credit: Konrad Jacobs, Wikimedia Commons, CC BY-SA 2.0



Georg Frobenius
1849 – 1917

image credit: Oberwolfach Photo Collection, public domain

Notes:

Notes:

Practice Session

- ▶ we explore the Markov Chain convergence theorem in python
- ▶ we use lazy Markov chains to guarantee aperiodicity
- ▶ we explore stationary states of reducible Markov chains

```
14-01 - Total Variation Distance and Stationary States of Markov Chains
June 14, 2023
Ingo Scholtes

In the second unit we calculate the total variation distance between visitation probabilities. We further use eigenstructure and eigenvalues of the transition matrix to calculate the stationary state of a Markov chain.

1 import numpy as np
2 import numpy.linalg as nla
3 import networkx as nx
4 import networkx.algorithms as nalg
5
6 from numpy.linalg import eigvals
7 import sys
8
9 plt.style.use('default')
10 plt.rcParams['figure.dpi'] = 100
11
12 We reuse the functions and the example from the previous notebook:
13
14 def transition_matrix(network, weights=None):
15     """
16     A transition matrix for a network. If weights are given, they are used as weights.
17     If not, all edges are assumed to have weight 1.
18     """
19     A = nx.adjacency_matrix(network, nodelist=sorted(network.nodes()))
20     A = A.toarray()
21     if weights is not None:
22         # We assume that the weights are given as a list of (source, target) pairs.
23         # We use the weights to calculate the transition probabilities.
24         for i, j in weights:
25             A[i, j] = weights[i, j]
26     A = A / A.sum(axis=1)
27     return A
28
29 def visitation_probabilities(network, initial_dist, t):
30     """
31     The visitation probabilities after t steps.
32     """
33     A = transition_matrix(network)
34     p_t = np.dot(initial_dist, A**t)
35     return p_t
```

practice session

see notebooks 15-01 and 15-02 in [gitlab repository](https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2) at
→ https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2

Notes:

Random walks and ranking Web pages

- ▶ in our motivating example of losing a drunk friend, we used a Markov chain to calculate **visitation probabilities**
- ▶ suggests an **algorithm to rank locations** based on stationary probability to find our friend
- ▶ for **Web search engines**, typical search query will yield hundreds of thousands of web pages that contain given search term
- ▶ how can we **rank web pages** based on the probability to find relevant information?
- ▶ idea: web pages that are more likely to be visited by a **random surfer** are more “relevant”



DALL-E image generated for prompt “searching a needle in a haystack”

image credit: www.bing.com

Notes:

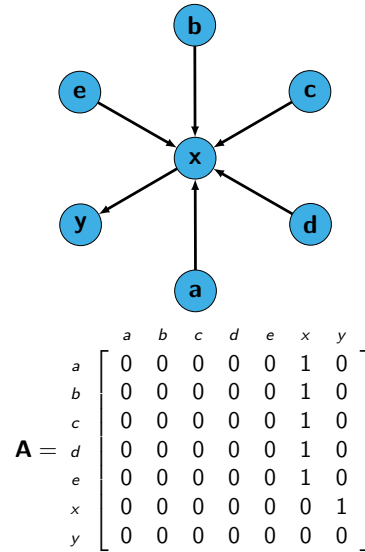
Modelling random surfers

- ▶ consider Markov chain model of **random surfer in Web graph** $G = (V, E)$, where V are web pages and $(v, w) \in E$ are **directed hyperlinks** between v and w
- ▶ for **random walk in directed graph**, we can define entries of transition matrix $\mathbf{T}$ as

$$T_{ij} = \frac{1}{\sum_j A_{ij}}$$

problems

- ▶ $\mathbf{T}$ may not be aperiodic
- ▶ $\mathbf{T}$ may not be irreducible
- ▶ T_{ij} not defined for nodes without outgoing edges



PageRank

- ▶ to ensure convergence to unique stationary state, we **include probability $1 - \alpha$** to teleport to node chosen uniformly at random (including node itself)
- ▶ with probability α random surfer continues to random neighbour (or random node if node has no neighbour)

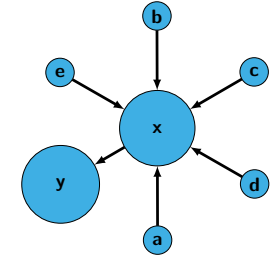
- ▶ we obtain new transition matrix with entries

$$T'_{ij} = \alpha \cdot T_{ij} + (1 - \alpha) \frac{1}{n}$$

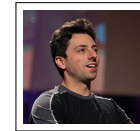
- ▶ we can calculate stationary state π of resulting Markov chain by solving **eigenvalue problem**

$$\pi = \alpha \pi \mathbf{T} + \vec{1} \frac{(1 - \alpha)}{n}$$

- ▶ entries of stationary state π are the **PageRank scores of nodes** → L Page, S Brin, 1998



PageRank score of nodes for $\alpha = 0.85$



Sergey Brin
born 1973

image credit: Steve Jurvetson,
Wikimedia Commons, CC-BY-SA



Larry Page
born 1973

image credit: Stansfield PL,
Wikimedia Commons, CC-BY-SA

Notes:

Notes:

Calculating PageRank for the Web?

- ▶ current Web graph has more than 3.35 billion nodes →

commoncrawl.org, December 2022

- ▶ more than 420,000 Gigabytes (including text content)

adjacency matrix of Web graph

- ▶ ≈ 9 Quintillion (10^{18}) entries!
- ▶ imagine you count 100 entries per second since the Big Bang
- ▶ you would still be counting today!

- ▶ how can we efficiently calculate (largest) eigenvalues and eigenvectors of a matrix?



Google's "data center" mid 1990s



power supply for one of Google's current data centers

image credit: Erik Pitti (top), Visitor7 (bottom), Wikimedia Commons, CC-BY-SA

Notes:

Eigenvalue algorithms and power iteration

- ▶ **efficient calculation of eigenvalues and eigenvectors** is important challenge in data science and AI

- ▶ **QR algorithm** builds on QR decomposition of matrix $A \in \mathbb{R}^{n \times n}$ → JGF Francis, 1961 and VN Kublanovskaja, 1961

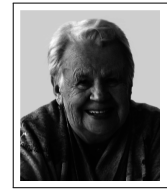
- ▶ but: complexity of QR-algorithm is $O(n^3)$

- ▶ we can use **iterative power iteration** to approximate largest eigenvector of a matrix

→ R von Mises and H Pollaczek-Geiringer, 1929

- ▶ starting with initial "guess" x_0 for eigenvector, **repeatedly multiply with matrix A and normalize**

- ▶ multiplication of vector with **sparse matrix** (i.e. with many zeros) is fast!



Vera Kublanovskaya, 1920 – 2012

image credit: public domain

power iteration

```
procedure POWER_ITERATION(A, iterations)
  x0 ← random(A.shape)
  for t in range(iterations) do
    xt+1 ← xt · A
    xt+1 ← xt+1 / ||xt+1||
  end for
  return xt
end procedure
```

Notes:

Practice Session

- ▶ we calculate the PageRank score of nodes in a directed graph
- ▶ we use power iteration to approximate leading eigenvector of a transition matrix

```
10-02: Stationary States of Random Walks
January 12, 2022
In the notebook we study the stationary state of random walks in networks. With this we will lay the foundation
for the analytical study of diffusion speed in the upcoming lecture.

1. Importing libraries and defining the graph
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
import random

# Create a directed graph with 6 nodes and 10 edges
G = nx.DiGraph()
G.add_nodes_from([0, 1, 2, 3, 4, 5])
G.add_edges_from([(0, 1), (0, 2), (1, 3), (2, 3), (3, 4), (4, 5), (5, 4), (4, 3), (3, 2), (2, 1)])

# Calculate the transition matrix
def transition_matrix(G):
    n = G.number_of_nodes()
    T = np.zeros((n, n))
    for i, j in G.edges():
        T[j, i] = 1
    # Normalize the matrix to get the transition matrix
    for j in range(n):
        out_degree = sum(T[j, i] for i in range(n))
        if out_degree > 0:
            T[j, :] = T[j, :] / out_degree
    return T

# Calculate the stationary state using power iteration
def stationary_state(T, n_iter=100):
    n = T.shape[0]
    x = np.ones(n) / n
    for i in range(n_iter):
        x = T.dot(x)
    return x

# Calculate the stationary state for the graph
T = transition_matrix(G)
x = stationary_state(T)

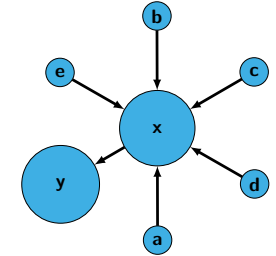
# Print the stationary state
print(x)
```

practice session

see notebook 15-03 in gitlab repository at
→ https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2

In summary ...

- ▶ **Markov chain convergence theorem** clarifies conditions under which Markov chain reaches **stationary state** in the limit of infinite time
- ▶ stationary states of random walks can be used to compute **relevance of nodes in a graph**
- ▶ rate at which nodes appear in close proximity on random walks can be used to define **similarity measure for nodes**



PageRank score of nodes for $\alpha = 0.85$

Notes:

Notes:

Exercise sheet 07

- ▶ **seventh exercise sheet** will be released today
- ▶ solutions are due **June 19th** (via WueCampus)
- ▶ present your solution to earn bonus points



Machine Learning for Networks

SoSe 2022

Prof. Dr. Ingo Scholtes

Chair of Informatics XV

University of Würzburg

Exercise Sheet 01

Published: May 4, 2022

Due: May 11, 2022

Total points: 70

Please upload your solutions to WueCampus as a scanned document (image format or pdf), a typesetted PDF document, or as a Jupyter notebook.

1. Computing connected components in graphs

(a) Implement Tarjan's algorithm to compute the (strongly) connected components in a (directed) network. Apply your algorithm to a directed example network that has multiple strongly connected components. 2P

(b) For a Laplacian matrix L of an undirected graph G with n nodes consider the sequence $\lambda_1 = 0 \leq \lambda_2 \leq \dots \leq \lambda_n$ of eigenvalues in ascending order. Generate a sequence of undirected networks with $n = 20$ nodes and different numbers of connected components from one to 20. Calculate the eigenvalue sequences of the corresponding Laplacians. What do you observe? Can you explain your observation? 1P

(c) Repeat the experiment above using the sorted sequence of eigenvalues of the adjacency matrix. 1P

(d) Use your finding from the previous tasks to implement a python function that uses the Laplacian matrix to calculate the number of connected components in an undirected graph. Test your function in an example network. What is the computational complexity of your function? 1P

2. Density-based Clustering with DBSCAN

Cluster detection, i.e., identifying groups of objects more similar to each other than to objects in other groups, is an important unsupervised machine learning task for collections of data points in a Euclidean space. Considering that similarities between points in Euclidean space can be represented by sets, graph-based algorithms have been successfully applied to this problem. A well-known example is DBSCAN, a density-based clustering algorithm that uses connected components in a graph to identify clusters based on contiguous regions with high point density. Consider the following paper (available on WueCampus) and answer the questions below.

M. Ester, H.P. Krögel, J. Sander, X. Xu: A density-based algorithm for discovering clusters in large spatial databases with noise. In: KDD'96: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining, pp. 226-231, August 1996.

(a) Give a pseudocode implementation of DBSCAN and explain the following aspects of the algorithm: 2P

- Explain how the parameter ϵ is used to represent the data points in terms of a graph.
- Explain how the parameter s influences the categorization of nodes as core, border, and noise nodes.
- Explain how we can apply Tarjan's algorithm to detect clusters.
- Discuss how the choice of the parameter s influences the number of detected clusters.

(b) Implement the algorithm in python and test it using synthetic data generated by the function `make_noise_problem` in `utils.py`. 2P

(c) Investigate for which values of ϵ the algorithm returns a reasonable cluster structure and compare the performance to k -means clustering (e.g. using the implementation included in `sklearn`). 1P

Self-study questions

1. For a real-square matrix **A**, what are the eigenvalues and eigenvectors of **A**?
2. Give an example for a Markov chain, where the transition matrix has degenerate eigenvalues.
3. Give an example for a graph in which a random walk never reaches a stationary distribution.
4. Give an example for a graph in which a random walk reaches multiple stationary distributions for different initial distributions.
5. Give an example for a transition matrix of a Markov chain where all states have a period of two.
6. How can a periodic transition matrix be made aperiodic?
7. When does a random walk in a network converge to a unique stationary distribution?
8. Given a transition matrix, how can we calculate the stationary state of a Markov chain?
9. What is the motivation for interpretation of PageRank as a measure of “relevance” for a Web page?
10. How is the transition matrix of the random walk model underlying PageRank defined?
11. Explain why a unique stationary state exists for the PageRank model, independent of the topology of the underlying graph.
12. Discuss how the parameter $\alpha \in [0, 1]$ influences the PageRank score of web pages. What do we get for $\alpha = 0$? What happens for $\alpha = 1$?

Notes:

Notes:

References

reading list

- ▶ O Perron: [Zur Theorie der Matrices](#), Math. Ann. 64, 1907
- ▶ G Frobenius: [Über Matrizen aus nicht negativen Elementen](#), Berl. Ber., 1912
- ▶ R von Mises and H Pollaczek-Geiringer: [Praktische Verfahren der Gleichungsauflösung](#), ZAMM - Zeitschrift für Angewandte Mathematik und Mechanik 9, 1929
- ▶ JGF Francis: [The QR Transformation, I](#), The Computer Journal, 1961
- ▶ VN Kublanovskaya: [On some algorithms for the solution of the complete eigenvalue problem](#), Journal of Computational Mathematics and Mathematical Physics, 1961
- ▶ S Brin, L Page: [The anatomy of a large-scale hypertextual web search engine](#), Computer networks and ISDN systems, 1998
- ▶ O Häggström: [Finite Markov Chains and Algorithmic Applications](#), 2002

248

OSCAR PERRON.

Zur Theorie der Matrices.

Von

OSCAR PERRON in München.

In dieser Note werden zum Teil bekannte Sätze aus der Theorie der Matrices und ihre charakteristischen Gleichung auf neue, höchst einfache Weise bewiesen, zum Teil neue Sätze entwickelt. Unter den Anwendungen der Theorie hebe ich ein der Gräffschen Methode analoges Verfahren zur näherungsweise Berechnung der Wurzeln einer algebraischen Gleichung hervor.

§ 1.

Sei

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}$$

das Koeffizientensystem einer linearen Substitution von n Variablen, E das der identischen Substitution. Soweit im folgenden nicht solche Koeffizientensysteme oder kürzer *Matrices* vorkommen, die rationale Funktionen von A , also miteinander vertauschbar sind, dürfen alle rationalen Rechenoperationen genau wie bei Zahlen ausgeführt werden. Bezeichnet man die Elemente der Potenz A^p mit $a_{ik}^{(p)}$, so daß also $a_{ii}^{(p)}$ mit a_{ii} gleichbedeutend ist, so ist wegen $A^{p+q} = A^p A^q$:

$$(1) \quad a_{ik}^{(p+q)} = \sum_{j=1}^n a_{ij}^{(p)} a_{jk}^{(q)}.$$

Konsequenterweise sollen auch die Elemente der Einheitsubstitution $E = A^0$ mit $a_{ik}^{(0)}$ bezeichnet werden; es ist also

$$a_{ik}^{(0)} = \begin{cases} 0 & \text{für } i \neq k, \\ 1 & \text{für } i = k, \end{cases}$$

und (1) gilt auch noch, wenn p oder q oder beide gleich Null sind. Sind $A = (a_{ik})$, $B = (b_{ik})$, $C = (c_{ik})$ beliebige Matrices, so sind die Gleichungen

$$A = B \quad \text{oder} \quad A = BC$$

Notes: